04 MAR 2026

AADK TASK 1

Accessibility Audit and Improvement Plan

MindMatrix

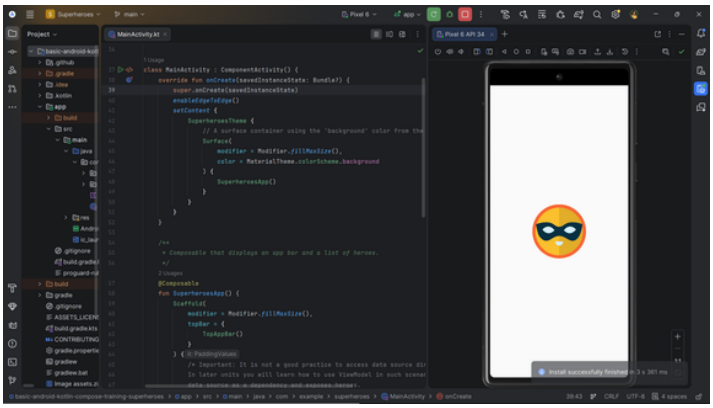
Selected Screen for Audit

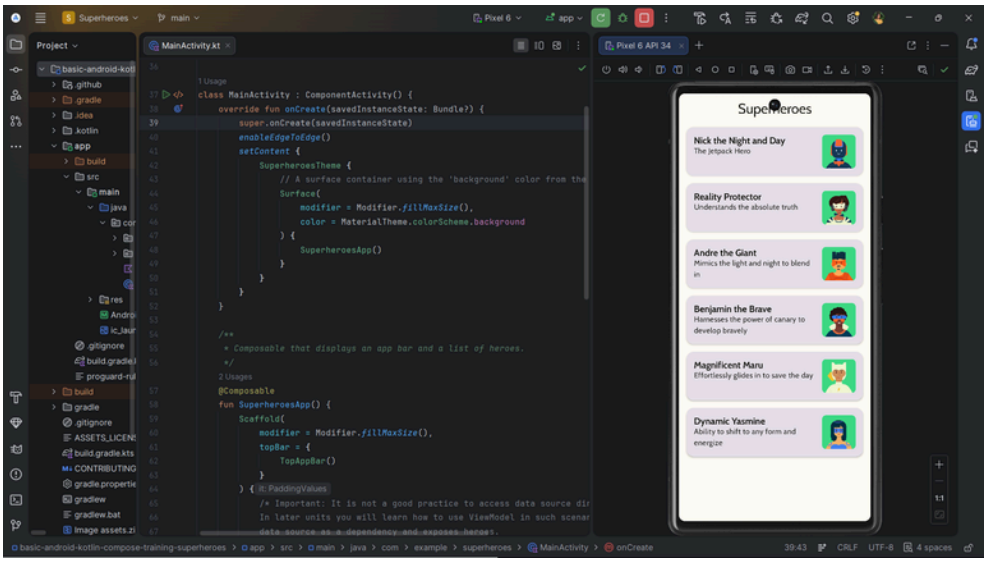
Screen Audited: Superhero List Screen

Description of Screen:

This screen displays:

- Top App Bar with title “Superheroes”
- LazyColumn list of superhero cards
- Each card contains:
  - Profile Image
  - Superhero Name
  - Short Description
  - “View Details” Button
- Simple fade-in animation on list items





Accessibility Audit Checklist

Accessibility Item	Status	Evidence / Observation
Content Description for Images	❌ Need Improvement	Avatar images do not have contentDescription.
Decorative Images Marked Null	❌ Need Improvement	Background decorative icon has description (should be null).
Touch Target Size ≥ 48dp	⚠️ Partial	“View Details” button is fine, but icon inside card is 32dp only.
Colour Contrast (WCAG AA 4.5:1)	❌ Need Improvement	Light grey text on white background has low contrast (~3.2:1).
Motion Reduction Support	❌ Need Improvement	Fade-in animation always runs; does not respect system reduce
Logical Focus Order	✅ Pass	Screen reader moves top-to-bottom logically.
Text Scalability (Large Font Test)	⚠️ Partial	When font scale is 1.5x, description text overlaps in some cards.
Clickable Elements Labeled Properly	⚠️ Partial	Button is readable, but entire card is clickable without clear label.
Screen Reader Testing (TalkBack)	⚠️ Partial	Reads text correctly, but does not announce card as group.

Improvement Plan

HIGH Priority Fixes

1. Add contentDescription to Meaningful Images

Problem: Profile images have no description.

Fix:

```
Image(
    painter = painterResource(id = hero.image),
    contentDescription = "${hero.name} profile picture"
)
```

2. Mark Decorative Images as Null

Problem: Decorative icon unnecessarily read by screen reader.

Fix:

```
Image(
    painter = painterResource(id = R.drawable.bg_icon),
    contentDescription = null
)
```

3. Fix Colour Contrast

Problem: Description text is too light.

Fix:

Replace:

```
color = Color.LightGray
```

With:

```
color = MaterialTheme.colorScheme.onSurface
```

Target contrast ratio: ≥ 4.5:1 (WCAG AA)

4. Support Reduce Motion Setting

Problem: Animation always plays.
Fix Plan:

- Use LocalViewConfiguration
- Detect animationScale
- Disable animation when system animation scale = 0

Example:

```
val scale = Settings.Global.getFloat(
    context.contentResolver,
    Settings.Global.ANIMATOR_DURATION_SCALE, 1f
)

if (scale != 0f) {
    // run animation
}
```

5. Ensure Minimum Touch Target Size

Problem: Icon size 32dp.

Fix:
Modifier.sizeIn(minWidth = 48.dp, minHeight = 48.dp)

6. Improve Card Semantics (Group Accessibility)

Problem: Icon size 32dp.

Fix:
Modifier.sizeIn(minWidth = 48.dp, minHeight = 48.dp)

7. Fix Large Font Layout Issues

Use:
maxLines = 3,
overflow = TextOverflow.Ellipsis

Also use:
Modifier.fillMaxWidth()

LOW Priority Fixes

- Add ripple indication to clickable cards
- Improve button accessibility label:

```
Modifier.semantics {
   contentDescription = "View details of ${hero.name}"
}
```

Integration with Automated Accessibility Tests

Testing Tools / APIs Used:

- Compose UI Testing
- SemanticsMatcher
- onNodeWithContentDescription
- assertIsDisplayed()
- assertHasClickAction()
- printToLog()

Proposed Test 1 – Image Has Content Description

```
@Test
fun heroImage_hasContentDescription() {
   composeTestRule
      .onNodeWithContentDescription("Iron Man profile picture")
      .assertIsDisplayed()
}
```

Proposed Test 2 – Button Is Accessible and Clickable

```
@Test
fun viewDetailsButton_isClickable() {
    composeTestRule
        .onNodeWithText("View Details")
        .assertHasClickAction()
}
```

Proposed Test 3 – No Missing Semantics

```
@Test
fun noSemanticsErrors() {
    composeTestRule
        .onRoot()
        .printToLog("AccessibilityTree")
}
```

5. Reflection

Accessibility is critical because mobile apps are used by people with different abilities, including visual, motor, and cognitive impairments. If we do not design for accessibility, we exclude a large group of users. Planning accessibility from the beginning makes UI cleaner, more structured, and more scalable. During this audit, I realized that small things like missing contentDescription or low colour contrast are very easy to overlook but can greatly affect usability. The biggest challenge I found was checking animation behavior against system motion settings, which is not automatically handled in Compose.

This task helped me understand that accessibility is not optional — it is a responsibility of every developer.